Uso de bases de datos no relacionales.

Las **bases de datos no relacionales (NoSQL)** ofrecen modelos de almacenamiento adaptados a aplicaciones que necesitan gestionar información diversa, evolucionar con rapidez y trabajar en entornos distribuidos.

Las aplicaciones actuales, como **redes sociales**, servicios de **streaming**, plataformas de comercio electrónico, aplicaciones móviles o sistemas en la **nube**, generan y consultan grandes cantidades de información de forma continua. Para responder a estas necesidades han adquirido especial importancia las **bases de datos NoSQL (Not Only SQL)**.

NoSQL engloba diferentes formas de almacenar y organizar los datos y amplía las alternativas disponibles frente al modelo relacional tradicional. Su utilización depende de las características de cada aplicación y de la forma en que esta necesita consultar, modificar y distribuir la información.

Freepik. *BD No Relacionales* (CC BY-NC-SA)

Las bases de datos **relacionales y NoSQL no son soluciones excluyentes**. Ambas pueden utilizarse de manera complementaria dentro de una misma aplicación, seleccionando en cada caso la tecnología más adecuada.

Ejemplo: una aplicación de comercio electrónico puede utilizar diferentes sistemas de almacenamiento para gestionar los **pedidos**, las **sesiones de usuario** o los **eventos de navegación**, según los requisitos de cada tipo de información.

? Preguntas introductorias

1. ¿Qué necesidades de una aplicación pueden hacer recomendable utilizar una base de datos **NoSQL**?
2. ¿Pueden utilizarse conjuntamente bases de datos **relacionales** y **NoSQL** en una misma aplicación?
3. ¿Es adecuado utilizar el mismo tipo de base de datos para almacenar **perfiles**, **sesiones**, **eventos** y **relaciones** entre usuarios?
4. ¿Qué aspectos deben analizarse antes de elegir una base de datos: **tipo de datos**, **acceso**, **rendimiento**, **escalabilidad** o **consistencia**?

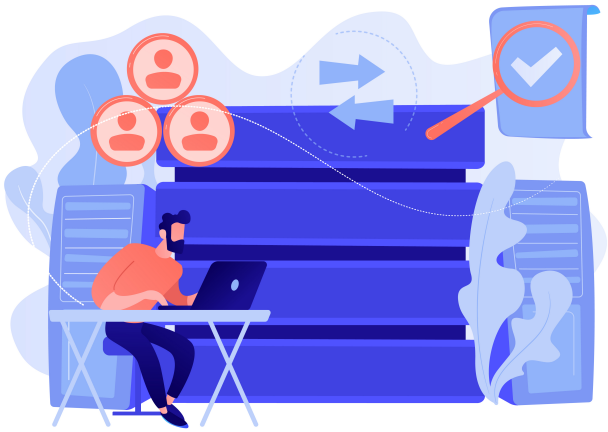
GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.
Aviso Legal

1. Bases de datos no relacionales y sus principales características.

Las **bases de datos no relacionales**, también conocidas como **NoSQL (Not Only SQL)**, son sistemas de almacenamiento de información diseñados para dar respuesta a las necesidades de las **aplicaciones modernas**, donde se manejan grandes volúmenes de datos, se requiere alta disponibilidad y es necesario adaptarse rápidamente a cambios en la estructura de la información.

Finalidad y para qué se utilizan las bases de datos no relacionales



Freepik. *BD No Relacionales* (CC BY-NC-SA)

Las bases de datos NoSQL se utilizan principalmente cuando:

- La estructura de los datos **cambia con frecuencia**.
- Se requiere **alta escalabilidad** para soportar muchos usuarios simultáneos.
- Se gestionan **grandes cantidades de datos no estructurados o semiestructurados**.
- Se necesita **rapidez en el acceso a la información**.
- La aplicación debe estar **siempre disponible**, incluso ante fallos parciales.

Ejemplos de uso habitual:

- Redes sociales (gestión de perfiles, publicaciones, relaciones).
- Aplicaciones móviles con backend en la nube.
- Sistemas de mensajería en tiempo real.
- Plataformas de comercio electrónico.
- Internet de las cosas (IoT).
- Análisis de grandes volúmenes de datos (Big Data).

Características de las bases de datos no relacionales

Las bases de datos NoSQL presentan características que responden a las necesidades de las aplicaciones actuales, como la gestión de información heterogénea, el crecimiento del volumen de datos, la necesidad de tiempos de respuesta reducidos y el funcionamiento en sistemas distribuidos.

Las características más relevantes que se desarrollarán a continuación son:

1. **Flexibilidad** del modelo de datos.
2. **Escalabilidad**.
3. Optimización del **rendimiento** según los patrones de acceso.

4. Funcionamiento en entornos **distribuidos**.
5. Diseño orientado al **uso de los datos**.
6. Integración con aplicaciones **web**, **móviles**, **cloud** e **IoT**.

1. Flexibilidad de los datos

Muchos sistemas NoSQL permiten utilizar un **esquema flexible**, de manera que la estructura de la información puede evolucionar sin obligar a que todos los registros tengan exactamente los mismos campos.

Esto facilita la adaptación de una aplicación cuando aparecen nuevos requisitos. Sin embargo, un esquema flexible no significa ausencia de control: algunos gestores permiten establecer **reglas de validación**, campos obligatorios o restricciones sobre los tipos de datos.

Ejemplo: en una aplicación, un usuario puede tener únicamente nombre y correo, mientras que otro puede incorporar además teléfono, dirección o preferencias. El documento actual utiliza este ejemplo para mostrar la flexibilidad estructural.

2. Escalabilidad

Muchos gestores NoSQL están diseñados para facilitar la **escalabilidad horizontal**, incorporando nuevos nodos o servidores cuando aumenta la cantidad de datos o la carga de trabajo.

Este enfoque permite repartir información y peticiones entre distintos equipos, por lo que resulta útil en aplicaciones que deben crecer progresivamente.

Ejemplo: una plataforma de *streaming* puede añadir nuevos servidores cuando aumenta el número de usuarios conectados.

La escalabilidad horizontal es habitual en sistemas NoSQL, aunque no es una característica exclusiva de ellos.

3. Rendimiento y acceso a los datos

Los sistemas NoSQL suelen estar optimizados para determinados **patrones de acceso**. Dependiendo del modelo utilizado, pueden recuperar información directamente mediante una clave, trabajar con documentos completos o recorrer relaciones entre datos.

Por tanto, el **rendimiento** depende de que el modelo elegido se adapte correctamente a las operaciones que debe realizar la aplicación.

Ejemplo: una aplicación que necesita comprobar constantemente una sesión de usuario puede utilizar un sistema optimizado para recuperar el dato mediante un identificador único.

4. Distribución y disponibilidad

Muchos sistemas NoSQL pueden funcionar de forma **distribuida**, utilizando varios nodos para almacenar o procesar la información.

Esta arquitectura facilita el crecimiento del sistema y permite utilizar mecanismos como el **particionado** y la **replicación** para distribuir los datos y mejorar la continuidad del servicio.

Las decisiones relacionadas con **consistencia**, **disponibilidad** y **tolerancia a particiones** se desarrollarán posteriormente en el apartado dedicado al **Teorema CAP**, por lo que no es necesario explicarlas aquí.

5. Diseño orientado al uso de los datos

En NoSQL es especialmente importante conocer cómo accederá la aplicación a la información antes de definir su estructura.

El diseño debe tener en cuenta los **patrones de acceso**, por ejemplo:

- Qué información se consulta conjuntamente.
- Qué operaciones son más frecuentes.
- Qué datos se leen o modifican habitualmente.
- Qué tiempo de respuesta necesita la aplicación.

Ejemplo: si una aplicación consulta normalmente el nombre, dirección y preferencias de un usuario al mismo tiempo, puede resultar conveniente organizar esa información de forma conjunta.

El documento ya recoge esta idea al indicar que el diseño NoSQL se relaciona con la forma en que la aplicación accede a los datos.

6. Integración con aplicaciones actuales

Las bases de datos NoSQL se utilizan habitualmente en aplicaciones y arquitecturas basadas en:

- Aplicaciones **web** y **móviles.**
- **APIs** y servicios backend.
- **Microservicios.**
- Plataformas **cloud.**
- Dispositivos **IoT.**
- Sistemas distribuidos y aplicaciones en **tiempo real**.

Ejemplo: una aplicación móvil puede utilizar una API para consultar y actualizar información almacenada en una base de datos NoSQL desplegada en la nube.

Capital Software. *Bases de datos relacionales y no relacionales* (CC BY-NC-SA)

Bases de datos no relacionales (NoSQL)

Característica	Ventajas principales	Ejemplo
Ausencia de esquema fijo (flexibilidad estructural)	**Adaptación a cambios**, **reducción del tiempo de desarrollo** y posibilidad de trabajar con **datos heterogéneos**	En **MongoDB**, un usuario puede tener solo **nombre** y **correo**, mientras que otro incluye **dirección** y **preferencias**
Escalabilidad horizontal	**Añadir nodos**, **soportar muchos usuarios simultáneos** y mejorar la **tolerancia a fallos**	Una **red social** añade servidores para gestionar picos de acceso sin detener el servicio
Alto rendimiento en lectura y escritura	**Baja latencia**, **rapidez en el acceso a la información** y mejor	**Redis** se utiliza como **caché** para datos consultados con

Característica	Ventajas principales	Ejemplo
	experiencia de usuario	frecuencia
Gestión de grandes volúmenes de datos (Big Data)	**Almacenamiento distribuido**, **crecimiento continuo** y **procesamiento eficiente** de datos masivos	**Apache Cassandra** almacena datos de **sensores** y **registros de eventos**
Modelo de consistencia flexible (Teorema CAP)	**Alta disponibilidad**, **funcionamiento continuo** y **tolerancia a fallos** en sistemas distribuidos	En una **red social**, una publicación tarda unos segundos en mostrarse en todos los dispositivos
Diseño orientado a la aplicación	**Consultas simples**, **menor complejidad** del sistema y mejor **rendimiento**	Una **aplicación móvil** guarda toda la información del usuario en un **único documento**
Integración con tecnologías modernas	**Compatibilidad** con **aplicaciones web y móviles**, **APIs REST**, **microservicios** y **cloud**	Una app web accede a la base de datos mediante una **API REST** usando **JSON**
Alta disponibilidad y tolerancia a fallos	El sistema sigue operativo ante la **caída de nodos**	Una plataforma en la **nube** mantiene el servicio activo aunque falle un servidor
Mayor velocidad de desarrollo	**Evolución rápida** de la aplicación sin modificar esquemas rígidos	Un proyecto añade nuevos campos a los datos de usuario sin detener el sistema

Debes conocer

Las **bases de datos NoSQL** no sustituyen a las bases de datos relacionales, sino que ofrecen una solución más adecuada para aplicaciones modernas que requieren flexibilidad, escalabilidad y alto rendimiento en entornos distribuidos

Recomendación de recursos y enlaces para ampliar contenidos

- MongoDB – Documentación oficial. Enlace.

- **Por qué consultarlo:**Es la referencia principal para comprender las **bases de datos orientadas a documentos**, la **ausencia de esquema fijo** y el **modelado flexible de datos**, aspectos clave de las bases de datos no relacionales.

- Redis – Documentación oficial. Enlace
  - **Por qué consultarlo:**Permite entender de forma clara el **alto rendimiento** y el funcionamiento de las bases de datos **clave-valor**, especialmente en aplicaciones que requieren **acceso rápido a la información**.
- Apache Cassandra – Documentación oficial. Enlace.
  - **Por qué consultarlo:**Es un buen ejemplo de base de datos diseñada para la **escalabilidad horizontal** y la **gestión de grandes volúmenes de datos** en sistemas distribuidos.
- Teorema CAP (IBM) Enlace
  - **Por qué consultarlo:**Ayuda a comprender el **modelo de consistencia flexible** y las decisiones de diseño que se toman en las bases de datos no relacionales en entornos distribuidos.

Ejercicio Resuelto

Explica qué significa que una base de datos no relacional tenga ausencia de esquema fijo y menciona una ventaja de esta característica.

Mostrar retroalimentación

Respuesta:

Significa que no es necesario definir previamente la estructura de los datos y que cada registro puede tener campos diferentes. Una ventaja es que permite modificar los datos fácilmente sin cambiar toda la base de datos.

Solución:

La **ausencia de esquema fijo** implica que no existe una estructura rígida predefinida de campos y tipos de datos. Esto permite almacenar información con diferentes estructuras dentro de la misma base de datos. Una de sus principales ventajas es la **flexibilidad**, ya que facilita la adaptación a cambios sin necesidad de rediseñar la base de datos.

¿Qué es la escalabilidad horizontal en las bases de datos no relacionales y por qué es importante en aplicaciones con muchos usuarios?

Mostrar retroalimentación

Respuesta:

Es la capacidad de añadir más servidores para repartir la carga. Es importante porque permite que la aplicación funcione bien aunque

aumente el número de usuarios.

Solución:

La **escalabilidad horizontal** consiste en aumentar la capacidad del sistema añadiendo más **servidores o nodos**, en lugar de mejorar un único servidor. Es importante en aplicaciones con muchos usuarios porque permite **distribuir los datos y la carga**, mejorar el rendimiento y aumentar la **tolerancia a fallos**.

Indica una base de datos no relacional y explica brevemente para qué tipo de aplicación se utiliza habitualmente.

Mostrar retroalimentación

Respuesta:

MongoDB se utiliza para aplicaciones web y móviles que necesitan guardar datos en formato flexible.

Solución:

Un ejemplo de base de datos no relacional es **MongoDB**, que se utiliza habitualmente en **aplicaciones web y móviles** donde se necesita almacenar datos de forma **flexible** mediante documentos, adaptándose fácilmente a cambios en la estructura de la información.

1.1. Comparación entre bases de datos relacionales y NoSQL.

Las bases de datos **relacionales** y **NoSQL** ofrecen formas diferentes de organizar y gestionar la información. Ningún modelo es mejor en todos los casos: la elección depende de los **datos**, las **consultas** y los requisitos de la aplicación.

macrovector. *Bases de datos* (CC BY-NC-SA)

Las bases de datos relacionales organizan la información principalmente mediante **tablas**, filas, columnas y relaciones entre entidades. Las bases de datos NoSQL utilizan distintos **modelos de datos**, como documentos, clave-valor, familias de columnas o grafos.

La principal diferencia no debe plantearse como una oposición entre tecnologías antiguas y modernas, sino como distintos enfoques para resolver problemas de almacenamiento y acceso a la información.

Principales diferencias

Aspecto	Bases de datos relacionales	Bases de datos NoSQL
Modelo de datos	Tablas relacionadas mediante claves.	Documentos, clave-valor, grafos, familias de columnas, entre otros.
Esquema	Normalmente se define explícitamente mediante columnas, tipos y restricciones.	Algunos modelos permiten mayor **flexibilidad estructural**, aunque también pueden establecer reglas de validación.
Relaciones	Se representan mediante claves y pueden consultarse mediante operaciones como **JOIN**.	Dependen del modelo utilizado; en grafos, por ejemplo, las relaciones forman parte explícita de la estructura.

Aspecto	Bases de datos relacionales	Bases de datos NoSQL
Consultas	Utilizan principalmente **SQL**, lenguaje estandarizado ampliamente soportado.	Utilizan lenguajes, comandos o **APIs** específicos según el gestor.
Transacciones	Disponen tradicionalmente de un amplio soporte de propiedades **ACID**.	El soporte depende del gestor; algunas bases NoSQL también ofrecen transacciones ACID.
Escalabilidad	Pueden utilizar escalabilidad vertical y, según el sistema, también **horizontal**.	Muchos gestores están diseñados desde su origen para facilitar la escalabilidad horizontal y la distribución entre nodos.
Consistencia	Las garantías dependen del gestor, la configuración y el tipo de operación.	También pueden ofrecer distintos niveles de **consistencia** según la tecnología y configuración.
Uso habitual	Sistemas transaccionales, facturación, gestión empresarial y consultas estructuradas.	Aplicaciones distribuidas, datos variables, baja latencia, grandes cargas de escritura o relaciones específicas.

Esta comparación debe entenderse de forma orientativa. El documento actual presenta correctamente la idea de que ambos modelos pueden coexistir y que la elección depende de las necesidades de la aplicación, aunque algunas diferencias se expresan actualmente de forma demasiado absoluta.

Ejemplo

Una plataforma de comercio electrónico podría utilizar una base de datos **relacional** para gestionar pedidos, pagos y facturas, mientras emplea una solución **NoSQL** para almacenar sesiones, perfiles con información variable o grandes cantidades de eventos de navegación.

En este caso no se sustituye un modelo por otro, sino que cada tecnología se aplica al problema para el que resulta más adecuada.

Elaboración propia. *BSRelacionales_NoRelacionales* (CC BY-NC-SA)

Con la imagen anterior puedes visualizar:

1. Las bases de datos relacionales ofrecen **estructura, consistencia y potencia en consultas complejas**.
2. Las bases de datos NoSQL proporcionan **flexibilidad, escalabilidad y alto rendimiento en entornos distribuidos**.

Ambos modelos responden a necesidades distintas y, en muchos sistemas actuales, **coexisten** dentro de una misma arquitectura, utilizando cada uno allí donde aporta mayor valor.

Autoevaluación

¿Qué implica que una base de datos no relacional no tenga un esquema fijo?

- ○ Que todos los registros deben tener los mismos campos
- ○ Que la estructura de los datos puede variar entre registros
- ○ Que no se pueden modificar los datos una vez almacenados

Incorrecto

Solución. Las bases de datos no relacionales permiten **flexibilidad en la estructura de los datos**, de modo que cada registro puede contener diferentes campos sin necesidad de modificar la base de datos completa.

Incorrecto

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

¿Cuál es la principal ventaja de la escalabilidad horizontal en las bases de datos no relacionales?

- ○ Aumentar la potencia de un único servidor
- ○ Reducir el tamaño de la base de datos
- ○ Añadir más servidores para repartir la carga del sistema

Incorrecto

Incorrecto

Correcto. La **escalabilidad horizontal** permite incorporar nuevos **nodos o servidores**, distribuyendo los datos y mejorando el **rendimiento** y la **tolerancia a fallos** en aplicaciones con muchos usuarios.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

¿Para qué tipo de aplicación es especialmente adecuada una base de datos no relacional como Redis?

- ○ Aplicaciones que requieren alta velocidad de acceso a datos

- ○ Aplicaciones que necesitan relaciones complejas entre tablas
- ○ Aplicaciones con datos estrictamente estructurados

Correcto. **Redis** es una base de datos **clave-valor** optimizada para ofrecer **alto rendimiento** y **baja latencia**, por lo que se utiliza habitualmente como sistema de **caché** o en aplicaciones en tiempo real.

Incorrecto

Incorrecto

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

¿Qué afirmación es correcta en relación con MongoDB, Redis, Apache Cassandra y el Teorema CAP?

- ○ MongoDB y Cassandra requieren un esquema fijo y priorizan siempre la consistencia inmediata.
- ○ Redis se utiliza principalmente por su alto rendimiento y Cassandra destaca por su escalabilidad horizontal y tolerancia a fallos.
- ○ El Teorema CAP indica que un sistema distribuido puede garantizar siempre consistencia, disponibilidad y tolerancia a particiones.

Incorrecto. Esta afirmación es incorrecta. **MongoDB** no requiere un **esquema fijo**, ya que es una base de datos orientada a **documentos** con estructura flexible. **Apache Cassandra** tampoco utiliza un esquema rígido y, además, en sistemas distribuidos **no prioriza siempre la consistencia inmediata**, sino la **disponibilidad** y la **tolerancia a fallos**.

Correcto. Esta afirmación es correcta. **Redis** es una base de datos **clave-valor** optimizada para ofrecer **alta velocidad** y **baja latencia**, muy utilizada como sistema de **caché**. **Apache Cassandra** está diseñada para trabajar en **entornos distribuidos**, permitiendo la **escalabilidad horizontal** y una alta **tolerancia a fallos**.

Incorrecto. Esta afirmación es incorrecta. El **Teorema CAP** establece que un sistema distribuido **no puede garantizar simultáneamente** las tres

propiedades: **consistencia**, **disponibilidad** y **tolerancia a particiones**. Por ello, las bases de datos no relacionales suelen priorizar dos de ellas según las necesidades de la aplicación.

Solución

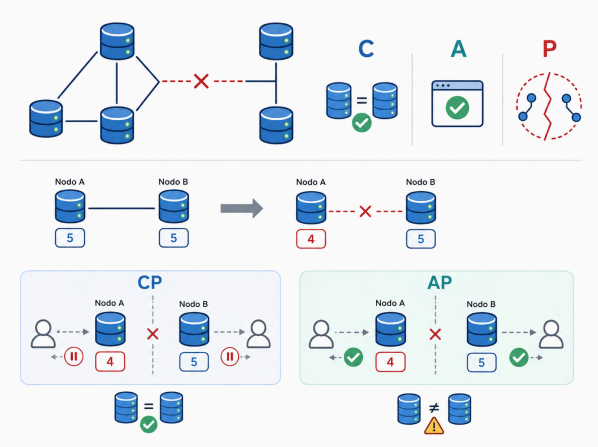
1. Incorrecto
2. Opción correcta
3. Incorrecto

1.2. Teorema CAP en sistemas distribuidos.

El **Teorema CAP** explica qué ocurre en un sistema distribuido cuando se produce una **partición de red** y algunos nodos dejan temporalmente de comunicarse.

CAP hace referencia a tres propiedades:

- **Consistencia (C)**: las lecturas deben ofrecer una visión coherente de los datos.
- **Disponibilidad (A)**: las peticiones reciben una respuesta mientras el nodo está operativo.
- **Tolerancia a particiones (P)**: el sistema puede seguir funcionando aunque haya problemas de comunicación entre nodos.



Elaboración propia. *Teorema de CAP* (CC BY-NC-SA)

En un sistema distribuido, cuando se produce una **partición de red**, no es posible garantizar al mismo tiempo una disponibilidad completa y determinadas garantías fuertes de consistencia. Por ello, el sistema debe establecer qué comportamiento prioriza en esa situación.

Comportamientos habituales

- **CP**: se prioriza la consistencia, aunque algunas operaciones puedan dejar de estar disponibles durante la partición.
- **AP**: se prioriza la disponibilidad, aunque temporalmente puedan existir diferencias entre las versiones de los datos.

No conviene clasificar de forma absoluta cada gestor como CP o AP, ya que el comportamiento puede depender de su **configuración**, arquitectura y tipo de operación.

Ejemplo

Dos nodos almacenan el mismo dato:

Nodo A: stock = 5
Nodo B: stock = 5

Si ambos dejan de comunicarse y el stock cambia en uno de ellos, el sistema debe decidir entre mantener la **consistencia** o seguir ofreciendo **disponibilidad** hasta que se restablezca la comunicación.

👀 Debes conocer

CAP no significa simplemente «elegir dos de tres» en cualquier situación. El compromiso aparece especialmente cuando existe una **partición de red**.

Ejercicio Resuelto:aplicación del Teorema CAP.

Situación

Una tienda online utiliza una base de datos distribuida con dos **nodos** que almacenan información sobre el stock de productos. Debido a un problema de red, ambos nodos dejan temporalmente de comunicarse.

Durante ese tiempo, varios usuarios siguen intentando consultar y comprar productos.

Pregunta

¿Qué decisión debe tomar el sistema según el **Teorema CAP** mientras exista la partición de red?

Mostrar retroalimentación

Durante una **partición de red**, el sistema debe decidir qué comportamiento priorizar:

1. Si prioriza la **consistencia**, puede bloquear determinadas operaciones hasta garantizar que los nodos trabajan con información coherente.
2. Si prioriza la **disponibilidad**, puede continuar respondiendo a las solicitudes, aunque temporalmente algunos nodos puedan manejar versiones diferentes de los datos.

La **tolerancia a particiones** debe mantenerse porque el problema de comunicación ya se está produciendo. Este es el compromiso fundamental que plantea CAP en sistemas distribuidos.

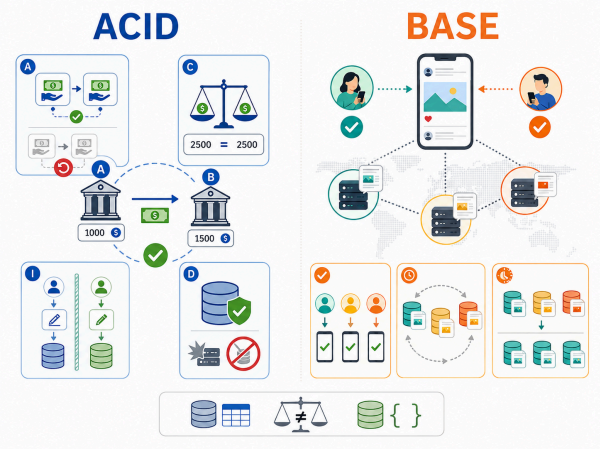
1.3. ACID, BASE y transacciones.

Las propiedades **ACID** y el enfoque **BASE** permiten comprender distintas formas de gestionar las operaciones, la consistencia y la disponibilidad de los datos. No deben asociarse de forma exclusiva a bases de datos relacionales o NoSQL.

Propiedades ACID

Una **transacción** es un conjunto de operaciones que deben gestionarse como una unidad lógica. Las propiedades ACID permiten realizar estas operaciones de forma fiable:

- **Atomicidad**: todas las operaciones de una transacción se completan correctamente o ninguna se aplica.
- **Consistencia**: la transacción mantiene las reglas de integridad establecidas.
- **Aislamiento**: las transacciones simultáneas no deben interferir entre sí de forma incorrecta.
- **Durabilidad**: una vez confirmados los cambios, estos permanecen almacenados aunque se produzca un fallo.



Elaboración propia. *ACID, BASE y transacciones* (CC BY-NC-SA)

Ejemplo: en una transferencia bancaria, retirar dinero de una cuenta y añadirlo a otra debe realizarse como una única transacción. Si alguna operación falla, el sistema debe evitar que la información quede en un estado incorrecto.

Enfoque BASE

El enfoque **BASE** se utiliza para describir determinados sistemas distribuidos que buscan mantener la disponibilidad y permiten que los datos alcancen progresivamente un estado coherente.

Sus principios pueden expresarse como:

- **Disponibilidad básica**: el sistema intenta continuar ofreciendo servicio.
- **Estado flexible**: los datos pueden cambiar mientras las actualizaciones se propagan entre los nodos.
- **Consistencia eventual**: si no se producen nuevos cambios, las distintas copias de los datos terminan siendo coherentes.

Ejemplo: en una aplicación social, una publicación puede aparecer primero en unos servidores y unos instantes después en otros, sin que la aplicación tenga que dejar de estar disponible.

ACID y BASE en las bases de datos actuales

No es correcto afirmar que las bases de datos relacionales son siempre ACID y las NoSQL siempre BASE.

Actualmente, algunos gestores NoSQL permiten utilizar **transacciones** y diferentes niveles de **consistencia**, por lo que las garantías ofrecidas dependen de la tecnología, la configuración y las necesidades de la aplicación.

Debes conocer

ACID define propiedades que aportan garantías a las transacciones, mientras que **BASE** describe un enfoque utilizado en determinados sistemas distribuidos que puede aceptar **consistencia eventual** para favorecer la disponibilidad.

Autoevaluación

Una aplicación bancaria realiza una transferencia de 200 € entre dos cuentas. El sistema descuenta el dinero de la primera cuenta, pero se produce un error antes de ingresarlo en la segunda. La operación debe cancelarse y dejar ambas cuentas como estaban antes de iniciar la transferencia.

¿Qué propiedad garantiza este comportamiento?

- ○ Durabilidad.
- ○ Atomicidad.
- ○ Consistencia eventual.
- ○ Disponibilidad básica.

Incorrecta. La durabilidad garantiza que una transacción ya confirmada permanezca almacenada aunque posteriormente se produzca un fallo.

Correcta. La atomicidad establece que una transacción debe completarse en su totalidad o cancelarse. Si una de sus operaciones falla, los cambios realizados deben deshacerse.

Incorrecta. La consistencia eventual indica que distintas copias de los datos pueden tardar un tiempo en alcanzar un estado coherente. Está relacionada con el enfoque BASE.

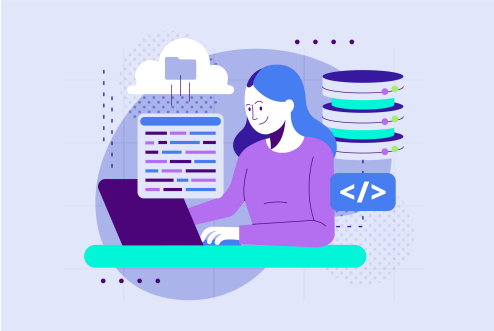
Incorrecta. La disponibilidad básica busca que el sistema continúe ofreciendo servicio, pero no garantiza que todas las operaciones de una transacción se completen o se cancelen conjuntamente.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2. Tipos de bases de datos no relacionales (NoSQL).

Las bases de datos **NoSQL** pueden clasificarse según la forma en que organizan y almacenan la información. Los modelos principales son **clave-valor**, **documentos**, **familias de columnas** y **grafos**.



Freepik. *Bases de datos no relacionales* (CC BY-NC-SA)

El modelo utilizado determina cómo se representan los datos y cómo se accede a ellos.

1. Bases de datos clave-valor

Almacenan la información mediante pares formados por una **clave única** y un **valor** asociado. La clave permite localizar directamente el dato, por lo que este modelo resulta sencillo y rápido.

Se utilizan habitualmente para **sesiones de usuario**, caché, configuraciones o información que debe recuperarse mediante un identificador.

Ejemplo: una aplicación puede almacenar una sesión utilizando como clave el identificador de sesión y como valor los datos asociados al usuario.

Tecnologías habituales: Redis, Amazon DynamoDB o Aerospike.

2. Bases de datos orientadas a documentos

Almacenan la información mediante **documentos** que contienen campos y valores. Estos documentos suelen representar una entidad completa y pueden incluir datos anidados.

Su estructura puede ser **flexible**, permitiendo que diferentes documentos de una misma colección tengan campos distintos.

Ejemplo: un usuario puede almacenarse en un documento con nombre, correo y teléfono, mientras que otro puede incluir además dirección y preferencias.

Tecnologías habituales: MongoDB, CouchDB o Firebase Firestore.

3. Bases de datos orientadas a familias de columnas

Organizan la información mediante **filas**, columnas y **familias de columnas**, y están diseñadas para trabajar con grandes cantidades de datos distribuidos entre varios nodos.

Son adecuadas para cargas elevadas de **lectura y escritura**, almacenamiento de eventos, telemetría o sistemas que necesitan crecer horizontalmente.

Ejemplo: una plataforma puede almacenar millones de eventos generados por dispositivos IoT distribuyendo los datos entre diferentes nodos.

Tecnologías habituales: Apache Cassandra, HBase, ScyllaDB o Google Bigtable.

> Conviene utilizar la expresión **familias de columnas** y no asociar este modelo directamente con las bases de datos columnares dedicadas principalmente al análisis de datos, ya que son conceptos diferentes.

4. Bases de datos orientadas a grafos

Representan la información mediante **nodos**, **relaciones** y propiedades. Su principal ventaja aparece cuando las conexiones entre los datos son una parte fundamental del problema.

Se utilizan en **redes sociales**, sistemas de recomendación, detección de fraude o análisis de relaciones.

Ejemplo: en una red social, los usuarios pueden representarse mediante nodos y las relaciones de amistad mediante conexiones entre ellos.

Tecnologías habituales: Neo4j y OrientDB.

Resumen

Tipo	Organización	Ejemplo de uso
Clave-valor	Clave y valor	Sesiones y caché
Documentos	Documentos con campos	Perfiles y aplicaciones web
Familias de columnas	Filas y familias de columnas	Eventos y grandes volúmenes de datos
Grafos	Nodos y relaciones	Redes sociales y recomendaciones

Debes conocer

No existe un modelo NoSQL adecuado para todas las situaciones. Cada tipo organiza la información de forma diferente y está orientado a determinados **patrones de acceso** y necesidades de la aplicación. Además, algunos gestores actuales pueden ofrecer características de más de un modelo.

Recomendación

Uso de tecnologías NoSQL: funcionamiento, objetivos y recursos de aprendizaje

Las **tecnologías NoSQL** proporcionan soluciones específicas para distintos tipos de necesidades en la gestión de la información. Cada sistema gestor incorpora **funcionalidades propias**, orientadas a un objetivo concreto, y dispone de **herramientas y documentación oficial** que facilitan su aprendizaje y uso.

Valor práctico de estas tecnologías

El uso de estas tecnologías permite:

- Seleccionar la **herramienta adecuada según el tipo de datos**.
- Optimizar rendimiento, escalabilidad y disponibilidad.
- Integrar la base de datos correctamente en aplicaciones reales.
- Aplicar buenas prácticas profesionales en la gestión de la información.

Tecnologías clave-valor.

Redis

Cómo se utiliza

- Se almacenan datos mediante **pares clave–valor**.
- El acceso a la información se realiza directamente a través de la clave.

- Permite definir **tiempos de expiración (TTL)** para datos temporales.
- Se utiliza tanto desde consola como mediante APIs desde aplicaciones.

Funcionalidades principales

- Gestión de sesiones de usuario.
- Caché de datos y resultados de consultas.
- Contadores, listas y colas en memoria.
- Persistencia opcional de los datos.

Objetivo principal

- Proporcionar **acceso extremadamente rápido** a información temporal o de uso frecuente.

Guías y tutoriales

- Documentación oficial: https://redis.io/docs
- Redis (formación y tutoriales gratuitos).
- Tutoriales y vídeos Redis.

Amazon DynamoDB

Cómo se utiliza

- Se definen tablas clave-valor totalmente gestionadas en la nube.
- El acceso se realiza mediante claves primarias y APIs.
- No requiere administración de servidores.

Funcionalidades principales

- Escalabilidad automática.
- Alta disponibilidad.
- Integración con servicios cloud.

Objetivo principal

- Gestionar grandes volúmenes de datos clave-valor de forma escalable en entornos cloud.

Guías y tutoriales

- Documentación oficial: https://docs.aws.amazon.com/dynamodb

Tecnologías orientadas a documentos

MongoDB

Cómo se utiliza

- Los datos se almacenan como **documentos JSON/BSON** en colecciones.
- Se pueden insertar, consultar, actualizar y eliminar documentos.
- Se gestiona mediante consola, interfaz gráfica o APIs.

Funcionalidades principales

- Estructura flexible sin esquema fijo.
- Índices avanzados.
- Consultas sobre campos anidados.
- Replicación y escalado.

Objetivo principal

- Almacenar información compleja y cambiante de forma flexible.

Guías y tutoriales

- Documentación oficial: https://www.mongodb.com/docs
- MongoDB Tutoriales:https://learn.mongodb.com

Firebase Firestore

Cómo se utiliza

- Se gestionan documentos en la nube desde aplicaciones web o móviles.
- Acceso directo mediante SDKs.
- Sin necesidad de administrar servidores.

Funcionalidades principales

- Sincronización en tiempo real.
- Escalabilidad automática.
- Integración con aplicaciones móviles.

Objetivo principal

- Facilitar el desarrollo rápido de aplicaciones con datos en tiempo real.

Guías y tutoriales

- Documentación oficial:https://firebase.google.com/docs/firestore

Tecnologías orientadas a columnas

Apache Cassandra

Cómo se utiliza

- Los datos se organizan en tablas orientadas a columnas.
- Se distribuyen automáticamente entre nodos.
- Se accede mediante lenguaje de consultas propio.

Funcionalidades principales

- Alta disponibilidad.
- Tolerancia a fallos.
- Inserciones masivas eficientes.

Objetivo principal

- Gestionar **grandes volúmenes de datos distribuidos** con alta disponibilidad.

Guías y tutoriales

- Documentación oficial:https://cassandra.apache.org/doc/latest

Tecnologías orientadas a grafos

Neo4j

Cómo se utiliza

- Los datos se representan como **nodos y relaciones**.
- Se realizan consultas centradas en las conexiones entre datos.
- Incluye interfaz gráfica para visualización de grafos.

Funcionalidades principales

- Consultas eficientes sobre relaciones complejas.
- Visualización gráfica de datos.
- Lenguaje de consultas específico.

Objetivo principal

- Analizar y gestionar **relaciones complejas** entre datos.

Guías y tutoriales

- Documentación oficial:https://neo4j.com/docs
- Neo4j GraphAcademy:https://graphacademy.neo4j.com

Tecnologías en memoria

Memcached

Cómo se utiliza

- Almacena datos en memoria mediante clave-valor.
- Se integra como caché entre la aplicación y la base de datos principal.

Funcionalidades principales

- Acceso ultrarrápido.
- Gestión de datos temporales.
- Arquitectura sencilla.

Objetivo principal

- Reducir la carga de la base de datos principal y mejorar tiempos de respuesta.

Guías y tutoriales

- Documentación oficial:https://memcached.org

Autoevaluación

Pregunta 1. Situación: gestión de sesiones

Una aplicación web con muchos usuarios necesita comprobar rápidamente si un usuario ha iniciado sesión cada vez que cambia de página. La información de la sesión es sencilla y se consulta constantemente.
¿Qué tipo de base de datos sería la más adecuada?

- ○ Base de datos orientada a documentos
- ○ Base de datos clave-valor
- ○ Base de datos orientada a grafos

Incorrecta. No es la opción más adecuada porque ofrece flexibilidad estructural, pero no es necesaria para datos simples como una sesión.

Correcta. Es la opción correcta porque permite un **acceso muy rápido** a datos sencillos mediante una clave, ideal para **sesiones de usuario**.

Incorrecta. Este tipo se utiliza cuando las **relaciones entre datos** son importantes, lo que no ocurre en este caso.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Pregunta 2. Situación: perfiles de usuario variables

Una aplicación móvil permite a los usuarios crear perfiles. Algunos usuarios solo introducen nombre y correo, mientras que otros añaden dirección, teléfono, intereses o preferencias.
¿Qué tipo de base de datos es la más adecuada para almacenar esta información?

- ○ Base de datos orientada a documentos
- ○ Base de datos en memoria
- ○ Base de datos orientada a columnas

Correcta. Es la opción correcta porque permite **estructuras flexibles**, donde cada usuario puede tener distintos campos sin modificar el esquema.

Incorrecta. Está pensada para datos temporales y muy rápidos, no para almacenamiento principal de perfiles.

Incorrecta. Se utiliza para grandes volúmenes de datos y análisis, no para perfiles individuales.

Solución

1. Opción correcta

2. Incorrecto
3. Incorrecto

Pregunta 3. Situación: análisis de grandes volúmenes de datos

Una empresa registra millones de eventos diarios (accesos, clics, errores) y necesita almacenarlos de forma distribuida para poder analizarlos posteriormente sin que el sistema deje de funcionar.

¿Qué tipo de base de datos sería la más adecuada?

- ○ Base de datos orientada a columnas
- ○ Base de datos en memoria
- ○ Base de datos clave-valor

Correcto. Es la opción correcta porque está diseñada para **Big Data**, **escalabilidad horizontal** y **alta disponibilidad**.

Incorrecto. No es adecuada para almacenar grandes volúmenes de datos de forma persistente.

Incorrecto. Ofrece rapidez, pero no está pensada para **análisis masivo de datos** ni consultas complejas.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

2.1. Criterios para la elección de una base de datos NoSQL según el contexto de uso.

La elección de una **base de datos NoSQL** no depende de que una tecnología sea mejor que otra, sino del **contexto de uso**, del **tipo de datos** y de las **necesidades de la aplicación**. Conocer y aplicar criterios adecuados permite seleccionar la solución más eficiente, escalable y coherente con los objetivos del sistema.

No existe una tecnología NoSQL adecuada para todas las situaciones. Antes de seleccionar un sistema es necesario analizar varios criterios. El PDF actual ya contempla el tipo de datos, crecimiento, acceso, consistencia y contexto tecnológico como factores de elección.

Freepik. *BDNoSQL* (CC BY-NC-SA)

Tipo y estructura de los datos

La forma de organizar la información es uno de los principales criterios de elección:

- **Clave-valor**: acceso principalmente mediante una clave.
- **Documentos**: información con estructura variable o jerárquica.
- **Familias de columnas**: grandes cantidades de registros distribuidos y cargas elevadas de escritura.
- **Grafos**: información donde las relaciones entre entidades son especialmente importantes.

Ejemplo: una aplicación móvil cuyos perfiles pueden contener campos diferentes puede utilizar una base de datos orientada a **documentos**.

Volumen de datos y crecimiento esperado

Debe valorarse tanto la cantidad actual de información como su posible crecimiento.

Cuando se espera un aumento importante del volumen de datos o de usuarios, es necesario considerar la **escalabilidad horizontal** y la posibilidad de distribuir la información entre varios nodos.

Ejemplo: un sistema que recibe millones de eventos de dispositivos puede necesitar una base de datos de **familias de columnas** preparada para crecer incorporando nuevos nodos.

Frecuencia y tipo de acceso

También debe analizarse cómo utilizará la aplicación los datos:

- Frecuencia de **lectura** y escritura.
- Necesidad de **baja latencia.**
- Consultas que se realizarán habitualmente.

- Forma en que se localiza la información.

Ejemplo: si una aplicación necesita comprobar constantemente una sesión mediante un identificador, un modelo **clave-valor** puede resultar adecuado. El PDF actual considera correctamente el patrón de acceso como un criterio específico de selección.

Consistencia y disponibilidad

La aplicación debe determinar qué garantías necesita respecto a la **consistencia** y la **disponibilidad** de la información.

No todas las aplicaciones tienen los mismos requisitos. Una operación económica puede requerir mayores garantías de consistencia, mientras que otros sistemas pueden aceptar que una actualización tarde un breve periodo en propagarse.

Este criterio está relacionado con **CAP**, **ACID** y **BASE**, estudiados anteriormente.

Contexto tecnológico y gestión

También deben valorarse aspectos relacionados con el entorno en el que se utilizará la base de datos:

- Compatibilidad con los **lenguajes** y tecnologías del proyecto;
- Disponibilidad de **controladores** y APIs;
- Opciones de administración y **monitorización**;
- Mecanismos de **seguridad** y copias de respaldo;
- Documentación, soporte y facilidad de mantenimiento.

Debes conocer

La elección de una base de datos NoSQL no debe realizarse únicamente por su popularidad o rendimiento. Es necesario analizar conjuntamente el **modelo de datos**, los **patrones de acceso**, el crecimiento previsto, las garantías necesarias y el contexto de la aplicación.

Autoevaluación

Una aplicación móvil permite que cada usuario tenga un perfil con campos distintos (nombre, correo, dirección, preferencias, etc.). **¿Qué tipo de base de datos NoSQL es más adecuada para este contexto?**

- ○ Base de datos clave-valor
- ○ Base de datos orientada a documentos

- ○ Base de datos orientada a columnas

Correcto: la base de datos orientada a documentos permite almacenar información con estructura flexible, adaptándose a perfiles de usuario variables sin necesidad de un esquema fijo.

Incorrecto: las bases de datos clave-valor están pensadas para datos simples identificados por una clave, no para información compleja.

Incorrecto: las bases de datos orientadas a columnas se utilizan principalmente para grandes volúmenes de datos y análisis masivo.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

Una aplicación web necesita comprobar constantemente si los usuarios tienen una sesión activa, priorizando la rapidez de acceso a la información. **¿Qué criterio ha sido determinante para elegir la base de datos NoSQL?**

- ○ El volumen de datos y crecimiento esperado
- ○ La frecuencia y tipo de acceso a los datos
- ○ La facilidad de gestión del sistema gestor

Incorrecto: el volumen de datos no es el factor principal cuando la necesidad es acceder rápidamente a información simple.

Correcto: la frecuencia y el tipo de acceso a los datos determinan la elección de bases de datos rápidas, como las clave-valor en memoria.

Incorrecto: la facilidad de gestión es importante, pero no es el criterio clave en este caso.

Solución

1. Incorrecto

2. Opción correcta
3. Incorrecto

En una red social se acepta que una publicación tarde unos segundos en mostrarse a todos los usuarios con tal de que el sistema siga funcionando sin interrupciones. **¿Qué criterio de elección se está priorizando?**

- ○ Consistencia estricta de los datos
- ○ Disponibilidad y tolerancia a fallos
- ○ Tipo de aplicación y contexto tecnológico

Incorrecto: la consistencia estricta implicaría que todos los usuarios vean la información al mismo tiempo.

Correcto: se prioriza la disponibilidad y la tolerancia a fallos, aceptando una consistencia eventual para garantizar el funcionamiento continuo del sistema.

Incorrecto: aunque el tipo de aplicación influye, no es el criterio principal en esta decisión.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

2.2. Casos prácticos de aplicación de bases de datos no relacionales.

La elección de un modelo **NoSQL** se comprende mejor analizando situaciones reales y relacionando las necesidades de la aplicación con la forma en que deben almacenarse y consultarse los datos.

Caso 1. Catálogo de productos con información variable

Freepik. *Aplicaciones Web* (CC BY-NC-SA)

Una tienda online dispone de productos con características diferentes. Un ordenador puede tener procesador y memoria, mientras que una prenda puede incluir talla, color y material.

Necesidad: almacenar productos cuya estructura puede variar según su categoría.

Solución: base de datos orientada a **documentos**.

Cada producto puede almacenarse en un documento con los campos que necesite, facilitando la incorporación de nuevas características.

Caso 2. Carrito de compra de acceso rápido

Una aplicación de comercio electrónico necesita consultar continuamente el carrito de cada usuario durante su navegación.

Necesidad: recuperar información rápidamente mediante un identificador único.

Solución: base de datos **clave-valor**.

El identificador del usuario o del carrito puede actuar como clave y los datos asociados como valor.

Caso 3. Registro de datos de dispositivos IoT

Una empresa recibe millones de mediciones procedentes de sensores: temperatura, consumo, fecha, dispositivo y ubicación.

Necesidad: realizar numerosas escrituras y almacenar grandes cantidades de información distribuida.

Solución: base de datos de **familias de columnas**.

Este modelo resulta adecuado para sistemas con grandes volúmenes de registros y crecimiento continuo.

Caso 4. Detección de relaciones entre operaciones

Una entidad necesita analizar conexiones entre usuarios, cuentas, dispositivos y operaciones para localizar posibles patrones de fraude.

Necesidad: recorrer y analizar numerosas relaciones entre entidades.

Solución: base de datos orientada a **grafos**.

Los usuarios, cuentas o dispositivos pueden representarse mediante nodos y sus conexiones mediante relaciones.

Caso integrador

Una misma aplicación puede utilizar diferentes modelos NoSQL cuando existen necesidades claramente distintas.

Por ejemplo, una plataforma de comercio electrónico podría emplear:

- **Documentos** para el catálogo de productos.
- **Clave-valor** para los carritos temporales.
- **Familias de columnas** para registrar eventos de navegación.
- **Grafos** para generar recomendaciones relacionadas con productos y usuarios.

Debes conocer

En una aplicación real no siempre es necesario utilizar un único modelo de base de datos. Lo importante es seleccionar el modelo que mejor se adapte a cada **tipo de información** y **patrón de acceso**.

Autoevaluación

Una plataforma registra millones de eventos diarios (accesos, clics y errores) y necesita almacenar esta información de forma distribuida sin que el sistema deje de funcionar cuando el volumen de datos aumenta. **¿Qué tipo de base de datos NoSQL es la más adecuada para este caso?**

- ○ Base de datos clave-valor
- ○ Base de datos orientada a grafos
- ○ Base de datos orientada a columnas

Incorrecto: las bases de datos clave-valor no gestionan relaciones complejas entre datos.

Correcto: las bases de datos orientadas a grafos permiten representar y consultar relaciones de forma eficiente.

Incorrecto: las bases de datos orientadas a columnas se utilizan principalmente para grandes volúmenes de datos y análisis.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Una aplicación web utiliza una base de datos orientada a documentos para perfiles de usuario, una clave-valor en memoria para sesiones y una orientada a columnas para eventos. **¿Qué ventaja principal aporta este uso combinado de bases de datos NoSQL?**

- ○ Reducir el número total de servidores del sistema
- ○ Adaptar cada base de datos a una necesidad concreta de la aplicación
- ○ Garantizar consistencia estricta en todos los datos

Incorrecto: el uso combinado no tiene como objetivo principal reducir servidores.

Correcto: permite utilizar el tipo de base de datos más adecuado para cada tipo de información y necesidad funcional.

Incorrecto: muchas bases de datos NoSQL priorizan disponibilidad y rendimiento frente a consistencia estricta.

Solución

1. Incorrecto
2. Opción correcta

3. Incorrecto

En una red social se necesita consultar rápidamente qué usuarios tienen amigos en común y generar recomendaciones basadas en relaciones. **¿Qué tipo de base de datos NoSQL se adapta mejor a esta necesidad?**

- ○ Base de datos clave-valor
- ○ Base de datos orientada a grafos
- ○ Base de datos orientada a columnas

Incorrecto: las bases de datos clave-valor no gestionan relaciones complejas entre datos.

Correcto: las bases de datos orientadas a grafos permiten representar y consultar relaciones de forma eficiente.

Incorrecto: las bases de datos orientadas a columnas se utilizan principalmente para grandes volúmenes de datos y análisis.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

3. Elementos de las bases de datos no relacionales.

Los sistemas **NoSQL** utilizan diferentes elementos para representar, organizar, localizar y distribuir la información. Estos elementos dependen del **modelo de datos** y del sistema gestor utilizado.

djvstock. *Bases de datos* (CC BY-NC-SA)

A diferencia de las bases de datos relacionales, no existe una única estructura común para todas las bases de datos NoSQL. Un sistema puede trabajar con documentos, claves o nodos, mientras que otros elementos permiten mejorar el acceso a la información o distribuirla entre varios servidores.

De forma general, podemos distinguir tres funciones:

- **Representación de datos**: elementos que permiten almacenar y describir la información.
- **Acceso y organización**: elementos utilizados para localizar y agrupar los datos de forma eficiente.
- **Distribución**: componentes que permiten repartir el almacenamiento y el procesamiento entre diferentes nodos.

Ejemplo: una aplicación puede almacenar los perfiles de usuario mediante documentos, utilizar un índice para localizar rápidamente un correo electrónico y distribuir la base de datos entre varios servidores cuando aumenta el volumen de información.

Debes conocer

Los elementos utilizados no son iguales en todos los sistemas **NoSQL**. Su significado y funcionamiento dependen del modelo y de la tecnología empleada.

Autoevaluación

Una aplicación utiliza diferentes tecnologías NoSQL. Para los perfiles emplea documentos, para localizar determinados datos utiliza índices y para distribuir la información trabaja con varios nodos.

¿Qué afirmación describe mejor los elementos de una base de datos NoSQL?

- ○ Todos los sistemas NoSQL utilizan exactamente los mismos elementos.
- ○ Los elementos utilizados dependen del modelo de datos y del sistema gestor.
- ○ Todas las bases NoSQL organizan la información mediante tablas y relaciones.
- ○ Los elementos NoSQL únicamente sirven para almacenar información.

Incorrecta. Los sistemas NoSQL pueden utilizar elementos diferentes según el modelo empleado.

Correcta. Cada modelo utiliza elementos propios para representar, organizar, localizar o distribuir la información.

Incorrecta. Las tablas y relaciones tradicionales pertenecen principalmente al modelo relacional. En NoSQL pueden utilizarse documentos, claves, nodos, relaciones u otras estructuras.

Incorrecta. Algunos elementos almacenan datos, pero otros permiten mejorar el acceso, organizar la información o facilitar su distribución.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

4. Incorrecto

3.1. Elementos: claves, valores, documentos, colecciones, nodos, relaciones, propiedades, índices y clústeres.

Las bases de datos **NoSQL** utilizan distintos elementos para representar, organizar y acceder a la información. Los elementos disponibles dependen del **modelo de datos** y del sistema gestor utilizado.

Freepik. *Bases de datos no relacionales* (CC BY-NC-SA)

No todos estos elementos aparecen en todas las bases de datos NoSQL. Algunos pertenecen a modelos concretos, mientras que otros, como los **índices** o los **clústeres**, pueden estar presentes en diferentes tecnologías.

1. Claves

Una **clave** es un identificador que permite localizar un dato de forma directa. Es un elemento fundamental en los sistemas clave-valor y también puede utilizarse para identificar o distribuir información.

Ejemplo: la clave `sesion_9845` permite recuperar los datos correspondientes a una sesión de usuario.

2. Valores

El **valor** es la información asociada a una clave. Puede ser un dato sencillo, como un número o texto, o una estructura más compleja.

Ejemplo: sesion_9845 → {usuario: 25, rol: "cliente"}

3. Documentos

Un **documento** agrupa los datos relacionados con una entidad mediante campos y valores. Puede contener información anidada y presentar una estructura flexible.

Ejemplo:

```
{
  "nombre": "Ana",
  "correo": "ana@ejemplo.com",
  "preferencias": ["libros", "tecnología"]
}
```

4. Colecciones

Una **colección** agrupa documentos relacionados dentro de una base de datos.

Los documentos de una misma colección pueden presentar diferencias en sus campos cuando el gestor permite un **esquema flexible**.

Ejemplo: una colección **usuarios** puede almacenar todos los perfiles de los usuarios de una aplicación.

5. Nodos

En una base de datos de grafos, un **nodo** representa una entidad del sistema, como una persona, producto, empresa o dispositivo.

Los nodos pueden contener propiedades y conectarse entre sí mediante relaciones.

Ejemplo: un nodo puede representar a un alumno con su nombre y curso.

6. Relaciones (aristas)

Las **relaciones** conectan los nodos y representan los vínculos existentes entre las entidades.

Pueden tener un tipo, una dirección y sus propias propiedades.

Ejemplo: ALUMNO ——MATRICULADO_EN——> MÓDULO

La relación puede incluir información adicional, como el curso académico.

7. Propiedades

Las **propiedades** almacenan información descriptiva asociada a los elementos de un grafo, especialmente a nodos y relaciones.

Ejemplo: un nodo que representa a un usuario puede tener propiedades como **nombre**, **ciudad** o **fechaRegistro**.

8. Índices

Un **índice** es una estructura auxiliar que permite localizar determinados datos con mayor rapidez.

Su utilización puede mejorar considerablemente el rendimiento de las consultas, aunque también requiere recursos adicionales y debe diseñarse según las búsquedas realizadas por la aplicación.

Ejemplo: crear un índice sobre el campo **correo** permite localizar rápidamente un usuario por su dirección de correo electrónico.

9. Clústeres

Un **clúster** está formado por varios servidores o nodos de almacenamiento que trabajan conjuntamente.

Puede utilizarse para repartir datos y carga de trabajo, facilitar la **escalabilidad** y mejorar la disponibilidad del sistema.

Ejemplo: si una aplicación aumenta considerablemente su volumen de información, el sistema puede distribuir los datos entre varios nodos del clúster.

Tecnologías de NoSQL

Elemento	Características clave	Tecnología de ejemplo
Claves	Identificación única y acceso rápido	Redis
Valores	Datos flexibles y variados	Redis
Documentos	Estructura flexible y completa	MongoDB
Colecciones	Agrupación sin esquema fijo	MongoDB

Elemento	Características clave	Tecnología de ejemplo
Nodos	Entidades del sistema	Neo4j
Relaciones	Conexiones entre entidades	Neo4j
Propiedades	Información descriptiva	MongoDB
Índices	Mejora del rendimiento	MongoDB
Clústeres	Escalabilidad y disponibilidad	Cassandrao en caché en memoria RAM, gratuito, de código abierto y de alto rendimiento, diseñado para acelerar aplicaciones web dinámicas

Debes conocer

Los términos **nodo** y **clúster** no deben confundirse. En un **grafo**, un nodo representa una entidad de los datos; en una arquitectura distribuida, un nodo puede ser también un servidor que forma parte de un clúster.

Recomendación

Estos recursos oficiales permiten profundizar en los **elementos de las bases de datos no relacionales** mediante tecnologías reales y actuales, facilitando la comprensión práctica de los conceptos estudiados.

Redis – Claves y valores

Elementos relacionados: Claves, ValoresRedis es un ejemplo claro de base de datos **clave-valor**, ideal para comprender cómo se utilizan las **claves como identificadores únicos** y los **valores como datos asociados**, destacando el acceso rápido y el alto rendimiento.

MongoDB – Documentos, colecciones, propiedades e índices

Elementos relacionados: Documentos, Colecciones, Propiedades, ÍndicesMongoDB permite entender de forma práctica el uso de **documentos con estructura flexible**, su organización en **colecciones sin esquema fijo**, la definición de **propiedades** y la creación de **índices** para mejorar el rendimiento de las consultas.

Neo4j – Nodos y relaciones

Elementos relacionados: Nodos, Relaciones

Neo4j es una base de datos orientada a grafos que facilita la comprensión del modelo basado en **nodos como entidades** y **relaciones como conexiones**, siendo especialmente útil para visualizar y consultar datos altamente relacionados.

Apache Cassandra – Clústeres

Elementos relacionados: ClústeresCassandra es un ejemplo representativo de base de datos distribuida, adecuada para entender cómo funcionan los **clústeres**, la **replicación de datos**, la **escalabilidad horizontal** y la **alta disponibilidad** en sistemas NoSQL.

Autoevaluación

Ejercicio 1. Gestión de sesiones de usuario

Situación:
Una aplicación web necesita comprobar de forma rápida si un usuario tiene una sesión activa. Para ello, se guarda un identificador único de sesión y los datos básicos del usuario.
¿Qué elemento de una base de datos no relacional es el más adecuado para identificar y acceder directamente a esa información?

- ○ Documento
- ○ Clave
- ○ Relación

Incorrecto. Un documento almacena información completa, pero no es el elemento principal para identificar y acceder rápidamente a un dato concreto.

Correcto. La clave permite **identificar de forma única** la información y acceder a ella de manera directa, siendo ideal para sesiones de usuario.

Incorrecto. Las relaciones se utilizan en bases de datos de grafos para conectar entidades, no para identificar datos.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Ejercicio 2. Almacenamiento flexible de perfiles

Situación:
En una aplicación móvil, cada usuario puede tener datos distintos (algunos solo nombre y correo, otros también dirección y preferencias). Toda la información de cada usuario se almacena junta.
¿Qué elemento de una base de datos no relacional permite almacenar esta información de forma flexible y estructurada?

- ○ Colección
- ○ Documento
- ○ Índice

Incorrecto. Una colección agrupa documentos, pero no es el elemento que almacena directamente los datos de cada usuario.

Correcto. El documento permite almacenar **toda la información de una entidad** con estructura flexible y sin esquema fijo.

Incorrecto. Un índice mejora el rendimiento de las búsquedas, pero no se utiliza para almacenar información.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Ejercicio 3. Búsqueda rápida de información

Situación:
Una base de datos contiene miles de registros de usuarios. La aplicación necesita buscar rápidamente a un usuario por su correo electrónico sin recorrer todos los datos.
¿Qué elemento se debe utilizar para mejorar el rendimiento de esta búsqueda?

- ○ Valor
- ○ Clúster
- ○ Índice

Incorrecto. El valor contiene los datos almacenados, pero no mejora por sí mismo la velocidad de las búsquedas.

Incorrecto. Un clúster mejora la escalabilidad y disponibilidad, pero no optimiza directamente las consultas.

Correcto. El índice permite **acelerar las consultas**, reduciendo el tiempo necesario para localizar información específica.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

3.2. Modelado NoSQL: diseño estructural de los datos.

El **modelado NoSQL** define cómo se organizarán los datos teniendo en cuenta tanto su estructura como la forma en que la aplicación necesita **consultarlos y modificarlos**.

A diferencia del modelado relacional, en NoSQL el diseño suele estar especialmente condicionado por los **patrones de acceso**. Antes de organizar los datos es necesario conocer qué información se consulta conjuntamente y qué operaciones serán más frecuentes.

Freepik. *Modelado NoSQL* (CC BY-NC-SA)

Organización de los datos

El diseño debe determinar:

- Qué datos deben almacenarse **juntos**;
- Cómo se representan las **entidades**;
- Qué campos necesita cada estructura;
- Qué información se consulta o modifica conjuntamente.

Ejemplo: Si una aplicación consulta habitualmente los datos personales, la dirección y las preferencias de un usuario al mismo tiempo, puede resultar conveniente almacenarlos conjuntamente en un mismo documento.

Datos agrupados o relacionados

En determinados modelos NoSQL puede decidirse entre almacenar información relacionada dentro de una misma estructura o mantenerla separada.

La decisión debe basarse en cómo utilizará los datos la aplicación:

- **Agrupar** información cuando se consulta normalmente de forma conjunta.
- Mantener información **separada** cuando tiene un ciclo de vida diferente, crece de forma independiente o se utiliza por separado.

Ejemplo: Los datos básicos y preferencias de un usuario pueden formar parte del mismo documento, mientras que un historial con millones de eventos puede almacenarse por separado.

Identificación y distribución de los datos

También es necesario definir correctamente los **identificadores** que permitirán localizar la información. En sistemas distribuidos, algunas claves pueden participar además en el **particionado** de los datos entre distintos nodos.

Una elección adecuada facilita consultas eficientes y evita concentrar excesivamente la carga en una parte del sistema.

Evolución del modelo

El diseño debe prever que la aplicación pueda evolucionar y necesitar nuevos campos o estructuras. La **flexibilidad** de muchos sistemas NoSQL permite introducir cambios progresivamente, aunque esto no elimina la necesidad de diseñar y documentar correctamente el modelo.

Debes conocer

En NoSQL no basta con decidir qué datos almacenar. Un buen modelo debe diseñarse pensando en los **patrones de acceso**, agrupando y organizando la información según la forma en que será utilizada por la aplicación.

Autoevaluación

Una aplicación web necesita mostrar en una sola consulta toda la información del usuario: datos personales, dirección y preferencias, ya que esta información se consulta constantemente. **¿Qué decisión de modelado NoSQL es la más adecuada?**

- ○ Separar la información del usuario en varias entidades independientes.
- ○ Almacenar toda la información del usuario en un único documento.
- ○ Guardar únicamente los datos básicos y consultar el resto en otra base de datos.

Incorrecto: separar la información obligaría a realizar múltiples accesos, reduciendo el rendimiento.

Correcto: almacenar toda la información en un único documento permite consultas rápidas y se adapta al modelo NoSQL orientado al uso de la aplicación.

Incorrecto: consultar otra base de datos aumenta la complejidad y el tiempo de respuesta.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Una aplicación gestiona sesiones de usuario que deben validarse rápidamente cada vez que se carga una página. Los datos son simples y temporales. **¿Qué modelo de datos NoSQL es el más adecuado para este caso?**

- ○ Modelo orientado a documentos
- ○ Modelo clave-valor
- ○ Modelo orientado a grafos

Incorrecto: el modelo orientado a documentos se utiliza para información más compleja y estructurada.

Correcto: el modelo clave-valor permite acceder rápidamente a datos simples mediante una clave única, ideal para sesiones.

Incorrecto: el modelo orientado a grafos se utiliza cuando las relaciones entre datos son prioritarias.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

En una red social se necesita consultar frecuentemente qué usuarios tienen amigos en común y analizar relaciones entre perfiles. **¿Qué decisión de modelado NoSQL es la más adecuada?**

- ○ Representar a los usuarios como documentos independientes sin relaciones.
- ○ Almacenar las relaciones en una tabla externa.
- ○ Representar usuarios como nodos y amistades como relaciones.

Incorrecto: sin relaciones explícitas, las consultas serían complejas y poco eficientes.

Incorrecto: el uso de tablas externas responde a un modelo relacional, no NoSQL.

Correcto: el modelado basado en nodos y relaciones permite gestionar y consultar conexiones complejas de forma eficiente.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

3.3. Gestión de la información según el modelo de datos NoSQL.

La gestión de la información en **NoSQL** depende del modelo utilizado. Cada modelo condiciona cómo se consulta, modifica, organiza y mantiene la información durante el funcionamiento de la aplicación.

Freepik. *NoSQL* (CC BY-NC-SA)

Gestión en bases de datos orientadas a documentos

La información se gestiona mediante **documentos** que pueden consultarse o modificarse completos o mediante campos concretos.

Durante el uso de la aplicación es habitual:

- Actualizar determinados **campos** de un documento;
- Incorporar nuevos campos cuando evoluciona la información;
- Consultar documentos mediante sus identificadores, campos o **índices**;
- Mantener agrupada la información que suele utilizarse conjuntamente.

Ejemplo: al modificar la dirección de un usuario, puede actualizarse únicamente ese campo dentro de su documento sin sustituir el resto de la información.

Gestión en bases de datos clave-valor

La información se gestiona principalmente mediante una **clave** que identifica el valor almacenado.

La aplicación debe controlar correctamente las claves utilizadas y, cuando los datos son temporales, su **tiempo de permanencia**.

Ejemplo: una sesión puede identificarse mediante una clave. Cuando la sesión deja de ser válida, el dato asociado puede eliminarse o caducar.

El acceso mediante una clave única constituye el mecanismo principal de gestión de este modelo en el documento.

Gestión en bases de datos de familias de columnas

La gestión debe tener en cuenta cómo se distribuyen los datos y cómo serán consultados. La elección adecuada de las **claves de partición** permite repartir la información entre diferentes nodos y acceder a ella de forma eficiente.

Ejemplo: los eventos generados por sensores pueden organizarse utilizando el dispositivo y un periodo de tiempo como parte de la estructura de acceso, evitando concentrar toda la información en un único nodo.

Gestión en bases de datos orientadas a grafos

La gestión se centra tanto en las entidades como en las **relaciones** existentes entre ellas.

Las relaciones pueden crearse, modificarse o eliminarse conforme cambia la información, y las consultas suelen realizar recorridos entre nodos conectados.

Ejemplo: cuando dos usuarios comienzan a seguirse en una red social, se crea una relación entre sus nodos. Si uno deja de seguir al otro, esa relación se elimina.

Mantenimiento de la información

Independientemente del modelo, la gestión debe procurar que los datos almacenados sigan siendo adecuados para las necesidades de la aplicación. Esto implica revisar aspectos como los **índices**, datos temporales, distribución de la información y evolución de sus estructuras.

Debes conocer

Gestionar información en NoSQL no consiste únicamente en almacenar datos. Es necesario adaptar su **acceso**, actualización y mantenimiento al modelo utilizado y a los **patrones de uso** de la aplicación.

Ejercicio Resuelto

Ejercicio 1. Gestión de perfiles de usuario

Situación

Una aplicación móvil gestiona perfiles de usuario. Algunos usuarios solo introducen nombre y correo electrónico, mientras que otros añaden dirección,

preferencias y un historial básico de actividad. La aplicación debe permitir añadir nuevos campos en el futuro sin modificar los perfiles existentes.

Pregunta ¿Qué modelo de datos NoSQL es el más adecuado para gestionar esta información y por qué?

Mostrar retroalimentación

El modelo orientado a documentos permite almacenar toda la información del usuario en un único documento con estructura flexible. Este modelo facilita la gestión de perfiles con campos variables y permite añadir nuevos datos sin redefinir un esquema fijo ni modificar los documentos existentes.

Ejercicio 2. Gestión de sesiones activas

Situación

Una aplicación web necesita comprobar constantemente si los usuarios tienen una sesión activa. Esta información se consulta en cada cambio de página y debe obtenerse con la menor latencia posible.

Pregunta ¿Qué modelo de datos NoSQL se debe utilizar para gestionar esta información y cómo se organiza?

Mostrar retroalimentación

El modelo clave-valor es el más adecuado para gestionar sesiones activas. Cada sesión se identifica mediante una clave única y el valor contiene los datos básicos de la sesión, lo que permite un acceso rápido y eficiente a la información.

Ejercicio 3. Gestión de relaciones entre usuarios

Situación

Una red social necesita gestionar las relaciones de amistad entre usuarios y permitir consultas como “usuarios con amigos en común” o “sugerencias de nuevos contactos”.

Pregunta ¿Qué modelo de datos NoSQL facilita este tipo de gestión de la información y cómo se representan los datos?

Mostrar retroalimentación

El modelo orientado a grafos facilita la gestión de relaciones complejas. Los usuarios se representan como nodos y las relaciones de amistad como conexiones entre ellos, permitiendo consultar y analizar fácilmente las relaciones entre usuarios.

4. Sistemas gestores NoSQL.

Un **sistema gestor NoSQL** es el software encargado de almacenar, consultar, administrar y proteger la información de una base de datos no relacional, actuando como intermediario entre los **datos** y las aplicaciones.

Freepik. *BD_SQL* (CC BY-NC-SA)

Estos sistemas están diseñados para dar soporte a **aplicaciones modernas**, donde se manejan grandes volúmenes de datos, se requiere un alto rendimiento y es necesario asegurar la disponibilidad del sistema en entornos distribuidos.

Funciones principales de un sistema gestor NoSQL

Almacenamiento y acceso a los datos

El gestor se encarga de almacenar la información según el **modelo de datos** utilizado y de proporcionar los mecanismos necesarios para recuperarla y modificarla.

Ejemplo: una aplicación puede solicitar al gestor los datos de un usuario mediante su identificador y recibir la información almacenada.

Distribución de la información

En sistemas distribuidos, el gestor puede repartir los datos entre diferentes **nodos** para soportar el crecimiento de la aplicación.

Esta distribución puede realizarse mediante técnicas de **particionado**, que dividen la información entre distintos nodos.

Ejemplo: cuando aumenta considerablemente el volumen de información, los datos pueden distribuirse entre varios servidores.

Replicación y disponibilidad

Algunos gestores permiten mantener varias **réplicas** de los datos en distintos nodos. Esto puede mejorar la disponibilidad del sistema y facilitar la recuperación ante determinados fallos.

No debe confundirse:

- **Particionado**: divide los datos entre diferentes nodos.
- **Replicación**: mantiene copias de los datos.

Seguridad y control de acceso

Los gestores NoSQL pueden proporcionar mecanismos de **autenticación**, usuarios, roles y permisos para controlar quién puede acceder a los datos y qué operaciones puede realizar.

Ejemplo: un usuario de la aplicación puede disponer únicamente de permisos de lectura, mientras que un administrador puede realizar operaciones de gestión.

Monitorización y mantenimiento

Un sistema gestor debe permitir conocer el estado de la base de datos y detectar posibles problemas de funcionamiento.

Puede proporcionar información sobre:

- Estado de los **nodos**;
- Utilización de recursos;
- Rendimiento de las consultas;
- Errores y disponibilidad del sistema.

Ejemplos de sistemas gestores NoSQL

Modelo	Sistema gestor
Documentos	MongoDB, CouchDB
Clave-valor	Redis, Amazon DynamoDB
Familias de columnas	Apache Cassandra, HBase
Grafos	Neo4j

Cada gestor dispone de características y mecanismos propios, aunque todos realizan la función general de gestionar el almacenamiento y acceso a la información.

Debes conocer

No debe confundirse el **sistema gestor** con las herramientas utilizadas para administrarlo. El gestor controla los datos y su funcionamiento; las consolas, interfaces gráficas, conectores o herramientas de monitorización permiten interactuar con él.

Autoevaluación

¿Es verdadero o falo?

Una aplicación aumenta rápidamente su número de usuarios. Para soportar la nueva carga, el sistema gestor permite incorporar nuevos servidores y distribuir los datos entre ellos.

Esta operación está relacionada con la escalabilidad horizontal de un sistema gestor NoSQL.

○ Verdadero ○ Falso

Verdadero

La escalabilidad horizontal permite aumentar la capacidad del sistema incorporando nuevos nodos y distribuyendo entre ellos los datos o la carga de trabajo.

Una empresa mantiene varias copias de determinados datos en distintos servidores para poder continuar ofreciendo el servicio si uno de ellos falla.

Este mecanismo se denomina particionado, porque consiste en crear varias copias de los mismos datos.

○ Verdadero ○ Falso

Falso

El mecanismo descrito es la replicación, ya que mantiene copias de los datos en diferentes nodos para mejorar la disponibilidad ante determinados fallos. El PDF contempla la replicación y la tolerancia a fallos entre las funciones del gestor.

En una base de datos NoSQL, el administrador dispone de permisos completos, mientras que un usuario de la aplicación únicamente puede consultar determinada información.

La gestión de usuarios, roles y permisos forma parte de las funciones que puede proporcionar un sistema gestor NoSQL.

○ Verdadero ○ Falso

Verdadero

Los gestores pueden incorporar mecanismos de control de acceso para determinar quién puede consultar, modificar o administrar la información.

5. Herramientas de los sistemas gestores de bases de datos no relacionales para la gestión de la información almacenada.

Los sistemas gestores **NoSQL** proporcionan diferentes herramientas para consultar, administrar, supervisar y proteger la información almacenada, tanto desde entornos de desarrollo como desde aplicaciones.

El uso adecuado de estas herramientas es fundamental para garantizar el **rendimiento**, la **seguridad** y la **disponibilidad** de la información en aplicaciones modernas.

1. Consolas de comandos

MongoDB. *MongoDB* (CC BY-NC-SA)

Las **consolas** permiten interactuar directamente con el gestor mediante instrucciones escritas. Son especialmente útiles para administración, pruebas y automatización de tareas.

Ejemplos: **mongosh** para MongoDB o **redis-cli** para Redis.

2. Interfaces gráficas de administración

Mongodbcompas. *Mongo DB compas* (CC BY-NC-SA)

Las **interfaces gráficas** permiten visualizar y administrar la información mediante un entorno visual, facilitando tareas como explorar datos, ejecutar consultas o gestionar estructuras.

Ejemplos: MongoDB Compass o Neo4j Browser.

3. Lenguajes y mecanismos de consulta

Cada gestor puede utilizar diferentes **lenguajes**, comandos o mecanismos para consultar y modificar la información.

Ejemplos:

- MongoDB utiliza su propia **API de consultas**.
- Cassandra utiliza **CQL**.
- Neo4j utiliza **Cypher**.

No es correcto afirmar que todos estos mecanismos simplemente «sustituyen a SQL». NoSQL no dispone de un único lenguaje de consulta estandarizado y cada tecnología puede adoptar una solución diferente.

4. Controladores y conectores

Los **controladores** y conectores permiten que una aplicación se comunique con la base de datos desde un lenguaje de programación.

Ejemplo: una aplicación Java puede utilizar un controlador de MongoDB para consultar y modificar documentos directamente desde su código.

5. Herramientas de seguridad

Permiten configurar mecanismos de **autenticación**, usuarios, roles y permisos para controlar el acceso a la información.

Ejemplo: asignar a una aplicación permisos de lectura y escritura únicamente sobre los datos que necesita.

6. Herramientas de monitorización

Permiten supervisar el estado y el **rendimiento** del sistema, detectar problemas y analizar el uso de recursos.

Pueden proporcionar información sobre consultas lentas, utilización de recursos, estado de los nodos o posibles errores.

Ejemplo: supervisar un clúster para detectar un nodo con una carga de trabajo anormal.

7. Copias de seguridad y restauración

Las herramientas de **copia de seguridad** permiten proteger la información frente a pérdidas y recuperar los datos cuando sea necesario.

Una copia de seguridad solo resulta realmente útil si existe un procedimiento que permita comprobar su **restauración**.

Freepik. *Bases de datos* (CC BY-NC-SA)

Resumen

Herramienta	Finalidad
Consola	Administración mediante comandos
Interfaz gráfica	Gestión visual de la información
Consultas	Recuperar y modificar datos
Controladores	Conectar aplicaciones con el SGBD
Seguridad	Gestionar accesos y permisos
Monitorización	Supervisar funcionamiento y rendimiento
Copias de seguridad	Proteger y recuperar información

👀 Debes conocer

Un mismo sistema gestor puede ofrecer varias herramientas de acceso y administración. La herramienta adecuada depende de la tarea que se quiera realizar: desarrollo, consulta, administración, seguridad, supervisión o recuperación de datos.

Autoevaluación

Pregunta 1. Gestión visual de datos

Situación:
Un desarrollador quiere **ver y modificar documentos almacenados** en una base de datos NoSQL de forma visual, sin escribir comandos, para comprobar rápidamente si los datos se guardan correctamente.
¿Qué herramienta del gestor NoSQL es la más adecuada?

- ○ Consola de comandos
- ○ Interfaz gráfica
- ○ Herramienta de copias de seguridad

Incorrecto. Permite gestionar los datos, pero requiere escribir instrucciones y no es la opción más cómoda para una visualización rápida.

Correcto. Es la herramienta adecuada porque permite **visualizar y modificar datos de forma intuitiva**, facilitando la administración.

Incorrecto. Se utiliza para proteger los datos, no para gestionarlos visualmente.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Pregunta 2. Integración con una aplicación

Situación:

Una aplicación multiplataforma necesita **acceder a la base de datos NoSQL desde el código**, realizando consultas y actualizaciones de forma automática.
¿Qué herramienta del sistema gestor se debe utilizar?

- ○ Lenguaje de consulta
- ○ API o conector
- ○ Herramienta de monitorización

Incorrecto. Permite consultar datos, pero no es el medio principal para integrar la base de datos dentro del código de la aplicación.

Correcto. Es la opción correcta porque permite **comunicar la aplicación con la base de datos**, realizando operaciones desde el programa.

Incorrecto. Se utiliza para supervisar el sistema, no para acceder a los datos desde una aplicación.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Pregunta 3. Mejora del rendimiento

Situación:
Una base de datos NoSQL funciona correctamente, pero algunas consultas tardan demasiado en devolver resultados. El objetivo es **mejorar el rendimiento** y detectar posibles problemas.
¿Qué herramienta del gestor NoSQL es la más adecuada?

- ○ Herramienta de monitorización
- ○ Interfaz gráfica
- ○ Herramienta de copias de seguridad

Correcto. Permite analizar el **rendimiento**, detectar consultas lentas y comprobar el uso de recursos.

Incorrecto. Facilita la gestión visual, pero no está orientada al análisis del rendimiento del sistema.

Incorrecto. Su función es proteger los datos, no mejorar la velocidad de las consultas.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

5.1. Puesta en marcha de un entorno MongoDB.

Para trabajar con **MongoDB** es necesario disponer de un servidor de base de datos y de una herramienta cliente que permita conectarse a él, como **mongosh** o MongoDB Compass.

Para poder trabajar con MongoDB se necesita un cliente (terminal o interfaz gráfico) y un servidor (local o remoto). Para ello podemos encontrar varias opciones:

msmk.university. *MongoDB* (CC BY-NC-SA)

Opciones para utilizar MongoDB

MongoDB Atlas permite crear una base de datos MongoDB en la **nube** sin instalar el servidor en el equipo. Una vez creado el entorno, se configura el acceso y se obtiene la información necesaria para realizar la conexión.

Una segunda posibilidad es instalar **MongoDB Community Edition** en un equipo local. Esta opción permite disponer del servidor directamente en Windows, Linux o macOS.

También puede utilizarse un **contenedor Docker**, lo que permite ejecutar MongoDB en un entorno aislado y fácilmente reproducible.

Acceso a MongoDB con mongosh

En una instalación local, el servicio de MongoDB debe estar iniciado antes de establecer la conexión. En sistemas Linux que utilizan **systemd**, puede iniciarse mediante:

```
sudo systemctl start mongod
```

Después se puede acceder al servidor con:

```
mongosh
```

Para comprobar las bases de datos disponibles puede utilizarse:

```
show dbs
```

Y para seleccionar una base de datos:

```
use registro_IoT
```

El comando **use** selecciona la base de datos indicada. MongoDB materializará la base de datos cuando se almacene información en ella. El ejemplo **registro_IoT** ya se utiliza en la práctica del documento.

Comprobación del entorno

Una vez conectado, puede comprobarse que el servidor responde correctamente mediante:

```
db.runCommand({ ping: 1 })
```

Una respuesta correcta confirma que el cliente puede comunicarse con el servidor MongoDB.

Entorno de trabajo

La estructura básica del entorno queda así:

Aplicación / usuario
↓
mongosh o MongoDB Compass
↓
Servidor MongoDB
↓
Base de datos
↓
Colecciones
↓
Documentos

Ejemplo

Para una aplicación que registra el estado de dispositivos **IoT**, se puede preparar un servidor MongoDB y seleccionar una base de datos denominada:

```
use registro_IoT
```

Posteriormente se podrán crear y gestionar los documentos correspondientes a los dispositivos.

Debes conocer

MongoDB puede ejecutarse de forma **local**, mediante **contenedores** o utilizando un servicio administrado en la **nube**. La elección del entorno no modifica el modelo documental, aunque sí cambia la forma de instalación, conexión y administración.

Autoevaluación

Una empresa quiere realizar pruebas con MongoDB en un equipo Linux. El servidor ya está instalado y el desarrollador necesita iniciar el servicio, acceder mediante terminal y comprobar las bases de datos disponibles.

¿Cuál es la secuencia más adecuada?

- ○ Ejecutar mongosh, después show dbs y finalmente iniciar mongod.
- ○ Iniciar el servicio mongod, acceder con mongosh y ejecutar show dbs.
- ○ Ejecutar show dbs, instalar MongoDB Compass y después iniciar mongosh.
- ○ Crear primero una colección y después conectarse al servidor.

Incorrecta. Antes de utilizar mongosh, el servidor MongoDB debe estar disponible para aceptar la conexión.

Correcta. Primero se inicia el servicio mongod, después se accede mediante mongosh y finalmente puede utilizarse show dbs para consultar las bases de datos disponibles.

Incorrecta. MongoDB Compass es una alternativa gráfica, pero no es necesario instalarlo para trabajar mediante terminal.

Incorrecta. Para crear colecciones o almacenar documentos es necesario conectarse previamente al servidor MongoDB.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

4. Incorrecto

5.2. Operaciones básicas de gestión de datos en bases de datos NoSQL.

La gestión de la información en una base de datos **NoSQL** se realiza mediante operaciones de creación, consulta, actualización y eliminación, conocidas habitualmente como operaciones **CRUD**.

Estas operaciones tienen la misma finalidad general en los distintos sistemas NoSQL, aunque se aplican sobre estructuras diferentes según el modelo: documentos, pares clave-valor, nodos y relaciones o registros organizados en familias de columnas.

Freepik. *Operaciones* (CC BY-NC-SA)

Creación de datos

La operación de **creación** permite incorporar nueva información a la base de datos.

Según el modelo utilizado, puede consistir en:

- Insertar un **documento.**
- Crear un par **clave-valor.**
- Añadir un **nodo** o una relación.
- Insertar un registro en una estructura de familias de columnas.

Ejemplo: cuando un usuario se registra en una aplicación, se crea la información necesaria para almacenar su perfil.

Consulta de datos

La operación de **lectura** permite recuperar la información almacenada.

Las consultas pueden realizarse utilizando identificadores, claves, campos concretos o relaciones, dependiendo del modelo NoSQL.

Ejemplo: una aplicación consulta el perfil de un usuario mediante su identificador para mostrar sus datos en pantalla.

Actualización de datos

La operación de **actualización** modifica información que ya existe.

La forma de realizarla depende de la estructura utilizada:

- Modificar uno o varios campos de un documento;
- Sustituir el valor asociado a una clave;
- Cambiar propiedades de nodos o relaciones.

Ejemplo: si un usuario modifica su teléfono, la aplicación actualiza ese dato manteniendo el resto de su perfil.

Eliminación de datos

La operación de **eliminación** permite borrar información que ya no debe mantenerse en la base de datos.

Puede implicar eliminar un documento, una clave y su valor, o determinados nodos y relaciones.

Ejemplo: cuando una sesión deja de ser válida, puede eliminarse la información asociada a ella.

Operaciones CRUD en una aplicación

En una aplicación real, las cuatro operaciones suelen utilizarse de forma combinada:

Operación	Ejemplo
Crear	Registrar un nuevo usuario
Leer	Consultar su perfil
Actualizar	Modificar sus preferencias
Eliminar	Borrar información que ya no es necesaria

Herramientas para realizar las operaciones

Las operaciones pueden ejecutarse mediante diferentes herramientas del sistema gestor:

- **Consolas de comandos.**
- **Interfaces gráficas.**
- Lenguajes o mecanismos de consulta.
- **APIs y conectores** desde el código de las aplicaciones.

Estas herramientas ya aparecen asociadas a las operaciones CRUD en el contenido actual.

Debes conocer

Las operaciones **CRUD** tienen una finalidad común en todos los sistemas NoSQL, pero su forma concreta de ejecución depende del **modelo de datos** y del sistema gestor utilizado.

Autoevaluación

Una aplicación móvil permite a un usuario añadir nuevas preferencias a su perfil sin modificar el resto de la información almacenada. **¿Qué operación básica de gestión de datos NoSQL se está realizando?**

- ○ Creación de datos
- ○ Consulta de datos
- ○ Actualización de datos

Incorrecto: la creación de datos se utiliza para añadir información nueva completa, no para modificar datos existentes.

Incorrecto: la consulta de datos solo permite recuperar información almacenada.

Correcto: la actualización de datos permite modificar campos concretos de un documento sin alterar su estructura completa.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

Una aplicación web necesita comprobar si un usuario tiene una sesión activa utilizando un identificador único y, en caso afirmativo, recuperar sus datos básicos. **¿Qué operación NoSQL se realiza principalmente en este proceso?**

- ○ Eliminación de datos
- ○ Consulta de datos
- ○ Creación de datos

Incorrecto: la eliminación de datos se realiza cuando la información deja de ser necesaria.

Correcto: la consulta de datos permite recuperar información almacenada utilizando una clave o identificador.

Incorrecto: la creación de datos se utiliza para insertar nueva información, no para verificar datos existentes.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Cuando un usuario cierra sesión en una aplicación, la información temporal asociada a su sesión se elimina de la base de datos. **¿Qué operación básica NoSQL se aplica en este caso?**

- ○ Actualización de datos
- ○ Eliminación de datos
- ○ Consulta de datos

Incorrecto: la actualización modifica datos existentes, pero no los elimina.

Correcto: la eliminación de datos permite borrar información que ya no es necesaria, como sesiones expiradas.

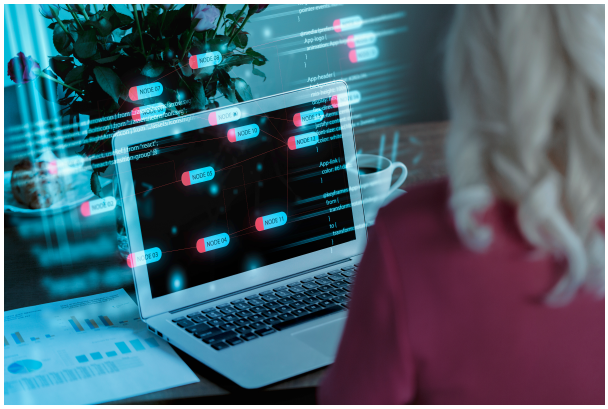
Incorrecto: la consulta de datos solo permite acceder a la información, no eliminarla.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

5.2.1.- Operaciones básicas de gestión de datos: CRUD en NoSQL y Fragmentación o particionado.

Las operaciones **CRUD** se realizan de forma diferente según el sistema NoSQL utilizado. Además, en sistemas distribuidos puede emplearse el **particionado** para repartir los datos entre distintos nodos.



magnific. *Gestión de datos* (CC BY-NC-SA)

En el punto anterior se ha explicado qué significan las operaciones de creación, lectura, actualización y eliminación. En este apartado se aplican mediante ejemplos concretos de diferentes gestores NoSQL.

CRUD en MongoDB

En MongoDB las operaciones se realizan sobre **documentos** almacenados en colecciones.

```
// Crear
db.usuarios.insertOne({
  nombre: "Ana",
  ciudad: "Sevilla"
})

// Leer
db.usuarios.find({
  nombre: "Ana"
})

// Actualizar
db.usuarios.updateOne(
  { nombre: "Ana" },
  { $set: { ciudad: "Córdoba" } }
)

```

```
// Eliminar
db.usuarios.deleteOne({
  nombre: "Ana"
})
```

CRUD en Redis

En Redis la información puede gestionarse mediante operaciones sencillas sobre pares **clave-valor**.

SET usuario:25 "Ana"

GET usuario:25

SET usuario:25 "Ana López"

DEL usuario:25

En este caso, **SET** puede utilizarse tanto para crear una clave como para modificar su valor.

CRUD en bases de datos de familias de columnas

En sistemas como Apache Cassandra se utilizan instrucciones adaptadas a su modelo de datos.

Ejemplos básicos:

```
INSERT INTO usuarios (id, nombre)
VALUES (1, 'Ana');

SELECT * FROM usuarios
WHERE id = 1;

UPDATE usuarios
SET nombre = 'Ana López'
WHERE id = 1;

DELETE FROM usuarios
WHERE id = 1;
```

CRUD en bases de datos de grafos

En Neo4j las operaciones se realizan sobre **nodos**, relaciones y propiedades utilizando Cypher.

// Crear

CREATE (u:Usuario {nombre: "Ana"})

// Leer

MATCH (u:Usuario {nombre: "Ana"})

RETURN u

// Actualizar

MATCH (u:Usuario {nombre: "Ana"})

SET u.ciudad = "Sevilla"

```
// Eliminar
MATCH (u:Usuario {nombre: "Ana"})
DELETE u
```

Fragmentación o particionado

El **particionado** consiste en dividir un conjunto de datos en partes y distribuirlas entre diferentes nodos o servidores. Su finalidad principal es facilitar la **escalabilidad** y repartir el almacenamiento o la carga de trabajo.

Ejemplo: una aplicación almacena millones de registros de dispositivos IoT. En lugar de concentrarlos en un único nodo, pueden distribuirse entre varios nodos utilizando una clave adecuada de partición.

Particionado y replicación

Es importante no confundir ambos conceptos:

Mecanismo	Finalidad
Particionado	Repartir diferentes partes de los datos
Replicación	Mantener copias de los datos

Ejemplo práctico: agenda en MongoDB

```
use agenda

db.createCollection("personas")
db.createCollection("citas")

db.personas.insertMany([
  {
    nombreUsuario: "juanillo",
    nombreCompleto: "Juan Pérez",
    edad: 23
  },
  {
    nombreUsuario: "mari",
    nombreCompleto: "María García",
    telefono: "987654321"
  }
])

db.citas.insertMany([
  {
    nombreUsuario: "juanillo",
    fecha: ISODate("2026-09-10"),
    hora: "10:00"
  },
  {
    nombreUsuario: "mari",
    fecha: ISODate("2026-09-11"),
    hora: "15:30"
```

```
  }
])
```

Para acelerar búsquedas frecuentes:

```
db.personas.createIndex({ nombreUsuario: 1 })
db.citas.createIndex({ fecha: 1 })
```

Ejercicio Resuelto

Crear en MongoDB una base de datos llamada "agenda" que contendrá una colección de "personas" y otra colección de "citas". Donde las personas tendrán como mínimo un campo "nombreUsuario" y otro "nombreCompleto" y las citas tendrán como mínimo un campo "nombreUsuario", "fecha" y "hora". Crear la indeces de los campos por los que se vaya consultar más frecuentemente. Introducir varias personas y citas de ejemplo. Crear consultas sobre personas por nombre y algún otro campo; y sobre citas para un "nombreUsuario" o unas "fecha" determinadas.

Mostrar retroalimentación

```
use agenda
db.createCollection("personas")
db.createCollection("citas")

// Insertar personas de ejemplo
db.personas.insertMany([
 { nombreUsuario: "juanillo", nombreCompleto: "Juan Pérez", edad: "23" },
 { nombreUsuario: "mari", nombreCompleto: "María García", telefono: "987654321" }
 ])

// Insertar citas de ejemplo
db.citas.insertMany([
 { nombreUsuario: "juanillo", fecha: ISODate("2024-05-07"), hora: "10:00" },
 { nombreUsuario: "mari", fecha: ISODate("2024-05-08"), hora: "15:30" }
 ])

// Creamos indices para agilizar la busqueda
db.personas.createIndex({ nombreUsuario: 1 })
db.personas.createIndex({ nombreCompleto: 1 })
db.citas.createIndex({ nombreUsuario: 1 })
db.citas.createIndex({ fecha: 1 })
```

```
// Consulta sobre personas
db.personas.find({ nombreUsuario: "juanillo" })
db.personas.find({ nombreCompleto: "María García" })

// Consulta sobre citas
db.citas.find({ nombreUsuario: "juanillo" })
db.citas.find({ fecha: { $in: [
 ISODate("2024-05-08"), ISODate("2024-05-09"), ISODate("2024-05-10")
 ] } })
```

Autoevaluación

Una plataforma de comercio electrónico utiliza distintas bases de datos NoSQL. En MongoDB necesita modificar la ciudad de un usuario ya existente. Además, el volumen de datos ha aumentado y se quiere repartir la información entre varios nodos para distribuir la carga.

¿Cuál de las siguientes opciones describe correctamente ambas acciones?

- ○ Utilizar insertOne() para modificar el usuario y aplicar replicación para repartir los datos entre nodos.
- ○ Utilizar updateOne() para modificar el usuario y aplicar particionado para distribuir los datos.
- ○ Utilizar find() para modificar el usuario y aplicar replicación para dividir los datos.
- ○ Utilizar deleteOne() para modificar el usuario y aplicar particionado para crear copias de los datos.

Incorrecta. insertOne() se utiliza para crear un nuevo documento. Además, la replicación mantiene copias de los datos, no los divide entre nodos.

Correcta. updateOne() permite actualizar un documento existente en MongoDB y el particionado permite dividir y distribuir los datos entre distintos nodos.

Incorrecta. find() se utiliza para consultar información. Además, la replicación no divide los datos, sino que mantiene copias.

Incorrecta. deleteOne() se utiliza para eliminar documentos. El particionado distribuye diferentes partes de los datos, no crea copias.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

5.3. Buenas prácticas en la gestión de la información almacenada.

Una gestión adecuada de la información **NoSQL** debe garantizar que los datos se mantengan seguros, accesibles, actualizados y con un rendimiento adecuado durante el funcionamiento de la aplicación.

Freepik. *Copias de Seguridad* (CC BY-NC-SA)

Las buenas prácticas no dependen únicamente del sistema gestor utilizado. También requieren controlar el acceso a los datos, revisar el rendimiento, disponer de mecanismos de recuperación y evitar la acumulación innecesaria de información.

Utilizar los índices de forma adecuada

Los **índices** pueden acelerar las consultas sobre campos utilizados frecuentemente, pero no deben crearse de forma indiscriminada.

Es recomendable:

- Crear índices sobre campos consultados habitualmente.
- Evitar índices que no aporten utilidad.
- Revisar periódicamente si siguen siendo necesarios.

Ejemplo: si los usuarios se buscan frecuentemente mediante su correo electrónico, puede resultar adecuado crear un índice sobre ese campo.

Controlar el acceso a la información

La base de datos debe disponer de mecanismos de **autenticación** y **autorización** que permitan controlar quién puede acceder a la información y qué operaciones puede realizar.

Se deben asignar usuarios, roles y permisos adecuados, evitando proporcionar más privilegios de los necesarios.

Ejemplo: la aplicación puede utilizar un usuario con permisos limitados, mientras que las tareas administrativas quedan reservadas a cuentas específicas.

Realizar copias de seguridad y comprobar su restauración

Las **copias de seguridad** permiten recuperar información ante errores, fallos o pérdidas accidentales.

Es recomendable:

- Programar copias periódicas.
- Almacenarlas en ubicaciones adecuadas.
- Comprobar que pueden **restaurarse** correctamente.

Ejemplo: realizar copias automáticas de una base de datos y probar periódicamente su restauración evita descubrir un problema con las copias cuando ya se ha producido una pérdida de información.

Supervisar el rendimiento y el estado del sistema

La **monitorización** permite detectar problemas antes de que afecten significativamente al funcionamiento de la aplicación.

Conviene controlar:

- Utilización de recursos.
- Consultas lentas.
- Errores frecuentes.
- Comportamiento durante periodos de alta carga.

Ejemplo: si una consulta comienza a tardar demasiado, la monitorización puede ayudar a determinar si es necesario revisar los índices utilizados.

Mantener la información actualizada y controlar su conservación

No todos los datos deben almacenarse indefinidamente. La información temporal, caducada o que ya no sea necesaria debe gestionarse mediante criterios de **retención**.

Ejemplo: las sesiones de usuario caducadas pueden eliminarse automáticamente para evitar la acumulación de información innecesaria.

Documentar la gestión de los datos

La **documentación** facilita el mantenimiento y la evolución de la aplicación, especialmente cuando participan varios desarrolladores.

Conviene documentar:

- La estructura de los datos.
- Los campos relevantes.
- Las decisiones adoptadas.
- Los procedimientos de administración y mantenimiento.

Ejemplo: indicar qué campos son obligatorios y cuáles opcionales facilita que otros desarrolladores puedan trabajar correctamente con la base de datos.

Debes conocer

Las buenas prácticas deben aplicarse durante todo el **ciclo de vida** de la información: controlar los accesos, optimizar las consultas, disponer de copias recuperables, supervisar el sistema y eliminar los datos que ya no sean necesarios.

Autoevaluación

Al diseñar un modelo de datos NoSQL, ¿qué criterio es más adecuado para organizar la información?

- ○ Normalizar los datos al máximo para evitar duplicaciones
- ○ Diseñar el modelo según la forma en que la aplicación utiliza la información
- ○ Utilizar la misma estructura para todos los registros

Incorrecto: la normalización estricta es propia del modelo relacional y no siempre es adecuada en NoSQL.

Correcto: diseñar el modelo según el uso real de la información permite mejorar el rendimiento y simplificar las consultas.

Incorrecto: obligar a todos los registros a tener la misma estructura reduce la flexibilidad del modelo NoSQL.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

¿Cuál es una buena práctica respecto al uso de índices en bases de datos NoSQL?

- ○ Crear índices sobre todos los campos para acelerar cualquier consulta
- ○ Evitar completamente el uso de índices
- ○ Crear índices solo sobre campos utilizados en búsquedas frecuentes

Incorrecto: un exceso de índices puede afectar negativamente al rendimiento de las escrituras.

Incorrecto: no usar índices puede provocar consultas lentas en bases de datos con muchos registros.

Correcto: los índices deben crearse únicamente en campos que se consultan con frecuencia para mejorar el rendimiento.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

¿Por qué es importante realizar copias de seguridad periódicas de la información almacenada?

- ○ Para reducir el tamaño de la base de datos
- ○ Para proteger la información frente a pérdidas o fallos del sistema
- ○ Para mejorar la velocidad de las consultas

Incorrecto: las copias de seguridad no tienen como objetivo reducir el tamaño de la base de datos.

Correcto: las copias de seguridad permiten recuperar la información en caso de errores, fallos técnicos o pérdidas accidentales.

Incorrecto: la velocidad de las consultas no depende directamente de las copias de seguridad.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

Licencia: licencia propietaria